CSE 375/475 - Fall 2026
Assignment 2: Concurrent Open-Addressed Hash Table
Due date: Friday, November 6

This assignment aims to implement three versions of an open-addressed hash set. To read more about open-addressed hash set design, you can find an introduction to that in Chapter 13.4 of the textbook "The Art of Multiprocessor Programming" (you can also check other books or online resources). The set implementation must provide the following APIs as described in the textbook: 'add', 'remove', and 'contains'. In addition to that, the set should provide a non-thread safe function (named 'size') that counts the number of elements in the hash set, and a non-thread safe function (named 'populate') that initializes the data structure with random n elements (n is a configuration parameter that can be used to test different initial sizes). The set must allow applications to use any generic (comparable) data type. Implementations that account for only integers are accepted with a penalty of 5 points.

The following three versions of an open-addressed hash table are required:

- A sequential version, as specified in Section 13.4.1.
- A concurrent version with a fixed number of locks, as specified in Sections 13.4.2 and 13.4.3.
- A concurrent version that uses the Transactional Memory abstraction as provided by GCC to convert the sequential version into concurrent.

In order to test the performance of the three versions, develop a simple application that spawns a pre-defined number of threads. Each of these threads executes a pre-defined number of operations on the data structure (or for a pre-defined time) with a pre-defined distribution of operations (e.g., 1M operations, 80% contains, 10% insert; 10% remove). Elements to be inserted, removed, or searched by these operations are random.

The outcome of each operation should be recorded so that the final size of the data structure at the end of the execution can be computed. When all threads are joined, the function ‘size’ should be invoked and it should match the value of the computed size.

When you perform your test, do not forget to populate your data structure first and then start benchmarking your application. The population time should not be part of the measured time.

The goal of the assignment is to contrast the performance of the two concurrent implementations (with the sequential version as a baseline to measure scalability). It is required to produce enough plots that show performance exploring different configurations.

Important notes about submission and grading

- It is REQUIRED to use the machines available at sunlab.cse.lehigh.edu. An evaluation performed on your own laptop will NOT BE considered sufficient unless a specific authorization has been granted by me.

- In order to achieve speedup, it is important to check that the machine you are logged in is not overcommitted. The sunlab machines can run 16 threads simultaneously ("less /proc/cpuinfo"), therefore if some other user is running along with you, you might not see the expected performance improvement. Use "top" (or alternative ways) to check the commitment of the machine you are using. If overcommitted, move to another machine. Remember that the "users" command shows the users currently logged in to the machine.

- The submission should be done through coursesite.lehigh.edu and should consist of:
    - The source code of your assignment
    - A document of 3-5 pages that summarizes your solution, evaluation, and analysis.

- Assignments will be graded during a one-on-one meeting with each student. You must be able to explain everything in your submission, including design attempts, plots, and analysis.

- The developing language is C++. Other languages cannot be used.

- TM is implemented in GCC starting from version 4.7. Use the latest version of GCC available in Sunlab.

Good luck!